

Lab 3 - Vectorisation

Advanced Computer Architectures

University of Minho

Lab 3 focuses on the vectorisation of a sample application on the A64FX CPUs. As usual, use a cluster node (through `sbatch` script submission) to run the analysis using the command line. The goal of Lab 3 is to evaluate the potential benefits of using vectorised code.

To load the compiler in the environment use these commands (use these or newer versions of each software):

GNU Compiler (for several system libraries): `module load GCC/13.3.0`

Consider the code available in the `/projects/F202500010HPCVLABUMINHO/ACA/lab3` folder in the Deucalion supercomputer.

1. Initial analysis of the code vectorisation on ARM devices

Lab 1.1 Analyse the source code files provided in the `lab3` folder, as well as the Makefile. The source code contains several functions that should, in theory, be vectorisable, and function skeletons to implement changes in case no vector code is generated.

Ensure that the code will be compiled for the A64FX CPU by adding the `-march=armv8-a` option in the Makefile. Note that the provided batch script compiles the code before executing, ensuring the compilation occurs in the A64FX compute node. Identify the vector instruction set used (SVE or NEON). Add the `-c` option to the compiler to analyse the assembly (in a `kernel.s` output file), if necessary. Also identify the vector width being used.

Lab 1.2 Probably, most functions were not vectorised. Analyse the vectorisation report by adding both of the following options to the compilation process, and resubmit the job:

`-fopt-info-vec` reports the code that was vectorised.

`-fopt-info-vec-missed` reports the code that was not vectorised, and possible reasons for that.

The report contains information about all compiled files. Focus only on the reports affecting `kernel.c`, and identify the functions that were not vectorised.

Lab 1.3 Force the use of SVE instruction set by using the `-march=armv8-a+sve` compiler option. Resubmit the job and confirm if the code is SVE by looking at its assembly. Can you identify the vector width being used?

Assess if the functions are still not vectorised, or if anything has changed from exercise 1.2.

2. Removing blockers to vectorisation

Lab 1.4 Analyse the comments in each function to be fixed, as they provide clues to the steps required to remove the blockers to vectorisation. Analyse and remove the blockers to vectorisation present in functions 1 to 3. Do not forget to submit the job script to correctly compile and run the code.

Remove the `-fvect-cost-model` option from the compilation process. Recompile and run the code. Were the initial non-vectorisable functions vectorised? If so, how and why? What does the cost model affect in the compilation process?

Lab 1.5 Consider function 4, which implements a sum of all elements in a vector. Implement this function using SVE intrinsics to feel the pain of not having the compiler do all the work for you. Check the function code for guiding comments.